

(Naive) Reuse Considered Harmful?

Reuse per boxes and lines is great—if you ignore operational needs!



Gregor Hohpe

I help architects connect the IT engine room with the executive penthouse through model thinking, decision discipline, and metaphors. I have experienced IT from virtually every angle: startups, Silicon Valley engineering, corporate IT, and consulting.

Follow

Related Posts

[Architecture is a game of constraint satisfaction](#)

[The Quest for Low-Code: 9 paths, some of which actually work](#)

[Architecture: Selling Options](#)

Popular Posts

[Would you like architects with your architecture?](#)

[Think like an architect](#)

[Agile and Architecture: Friend, not Foe](#)

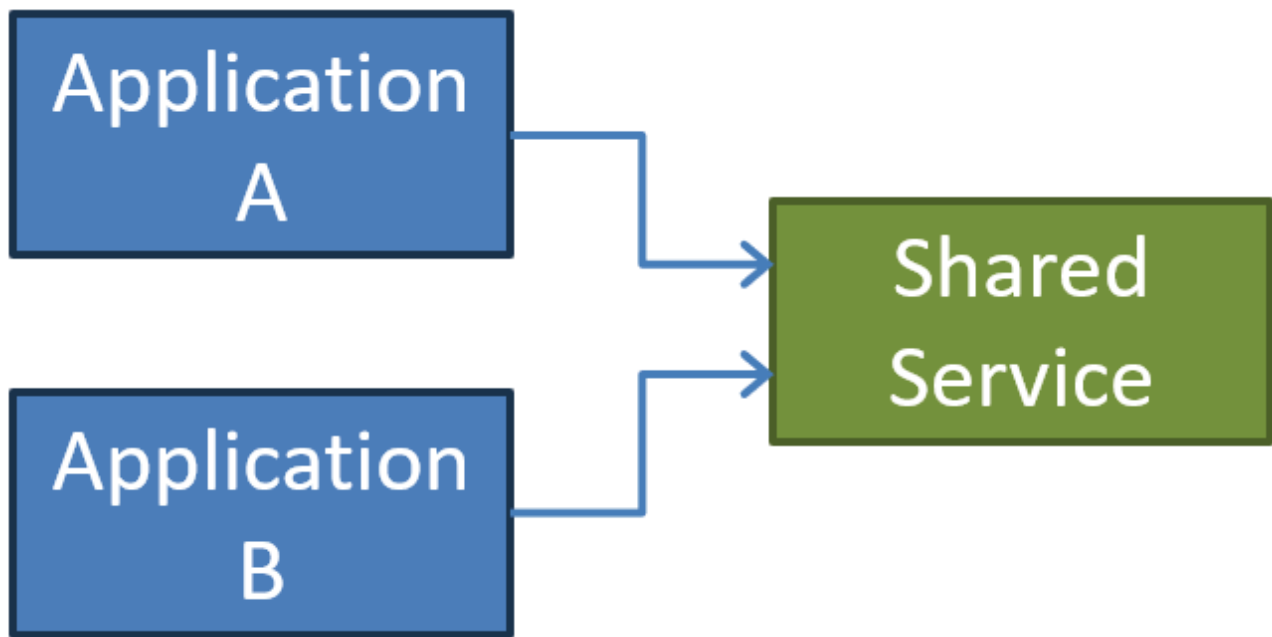


Updated: March 01, 2025



Category: [Architecture](#)

Architects, what is wrong with the following picture?



Isn't this exactly what all architects aim for? A shared service that is utilized by multiple applications, reducing diversity, complexity, and cost?

While many architects are already thinking about common APIs, authentication schemes, and complexity metrics, those of us who have been around large enterprises for some time know that the picture only tells part of the story. And the other part causes all the problems.

Static models don't tell a dynamic story

My book [Platform Strategy](#) highlights how static models cannot explain differences in behavior. In the book's case, platforms and traditional shared services layers look the same if you only consider the structure. They look very different, though, once you incorporate feedback loops and friction.

[The Software Architect Elevator](#) also picks on static diagrams, particularly capability diagrams, that don't include lines. The picture above at least includes lines, but the lines indicate a rather static property: dependency (or use), meaning Application A and Application B both use the shared service. Looks great, right? In principle yes, but it ignores the operational aspects of this dependency, something that is almost guaranteed to come back to haunt you.

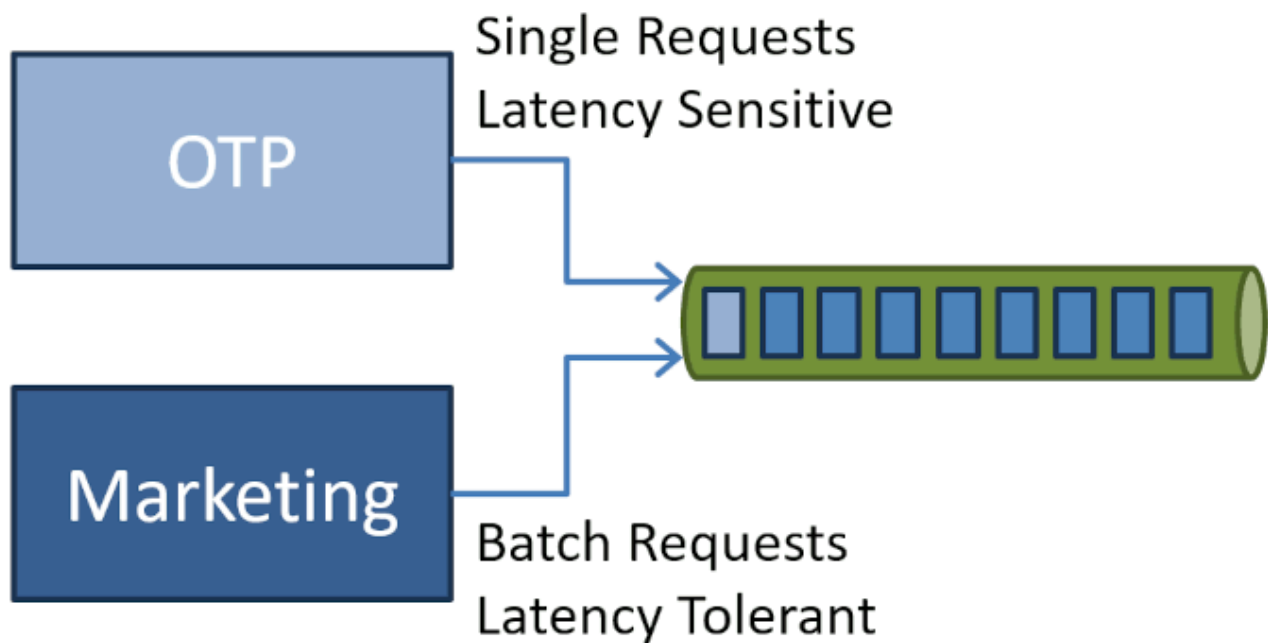
One-Time Passwords vs. Mass Mailing

I have this very “pattern” cause issues at two different banks, and I am sure it isn’t limited to banks. Both times, the reuse was related to sending one-time passwords (OTP) to users’ phones via SMS to enable cardless cash withdrawal at ATMs.



A bank in Southeast Asia used a shared component for SMS delivery to its customers. The component was developed by a well-known system integrator (names hidden to protect the guilty) and used across multiple use cases to avoid developing separate messaging systems. The setup headed my way when the customer (or rather the integrator—Asian banks tend to be at the mercy of their integrator) asked for another quota increase on the Amazon SNS text messaging service, 3 months after the last increase.

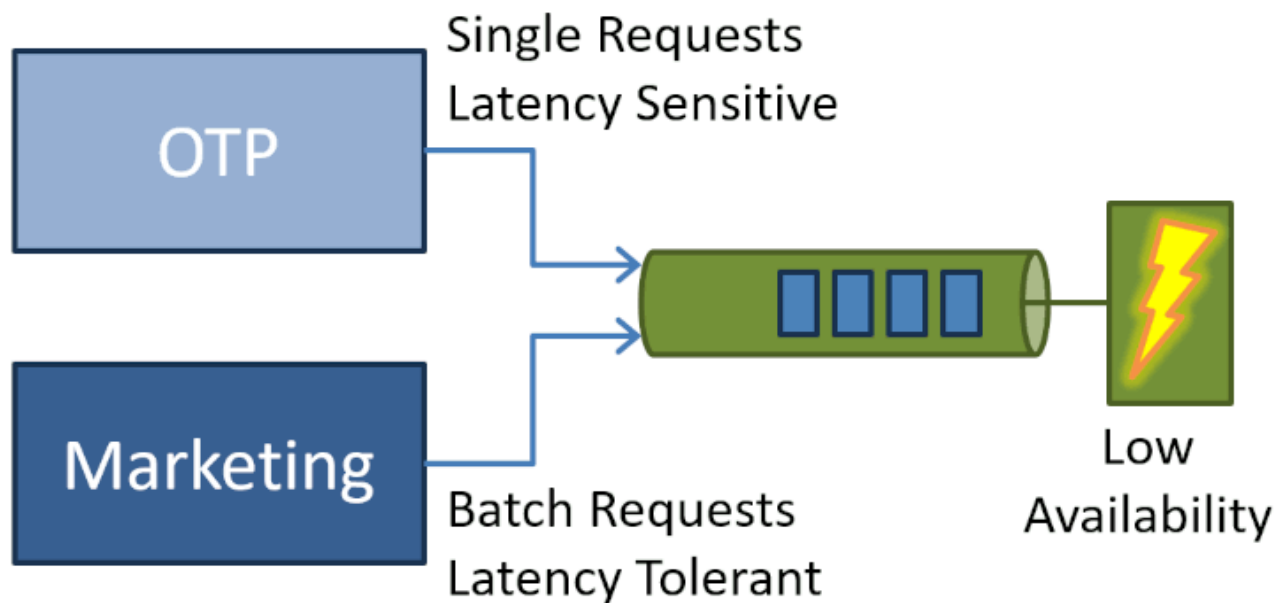
Sending mass-SMS isn’t cheap (about 1-2 cents per message) and also controlled to avoid spamming, so I was curious why they needed to send so many messages per second (and also whether users want to receive that many). During a call with the integrator, the problem became apparent:



The integrator used the same service instance (and quota) for both OTP and marketing messages. The operational needs of either use case are very different: OTPs require short latency (a few seconds) as they time out and users wait for them. The OTP traffic may have spikes during busy times, but consists of individual requests. Marketing messages are the exact opposite: they come in bulk and aren't latency sensitive at all. If a customer gets a promotional message 5 minutes later, it really doesn't matter (the customer may secretly enjoy it).

By favoring "reuse", it's easy to see what happened: when a marketing campaign runs, a giant batch of marketing messages clogs the pipe. OTP messages are then stuck behind all the marketing messages, causing delayed delivery and frustrated customers. The Band-Aid of increasing quota had been applied several times, of course never really addressing the design problem.

I had almost forgotten about it, until another bank (I tend to work with a lot of banks) had a similar issue. Marketing had built an SMS delivery service, but reliability wasn't a key criterion—if the service was down for a few minutes, no harm was done. Another part of the business needed OTP for two-factor authentication and quickly spotted the available service. The rest is history, so to speak, as the low reliability requirements caused many headaches for the OTP delivery.



The setup went into the IT annals as “misguided reuse”.

Duplication in the cloud isn’t duplication

Once you look at the problem in isolation, the solution options become apparent. For one, you can throttle the delivery of marketing messages to the shared service to a rate well below the available quota. Doing so will always leave room for OTP messages. If you want to avoid the small penalty of underutilizing the service quota when there are no OTPs, you could use a back-pressure approach: marketing messages are slowed down or stop whenever the queue size exceeds a certain threshold. You need to watch out that the available quota is sufficient for OTP spikes to avoid customers timing out, and also make sure that there is quota left for marketing messages. If marketing always backs off, the algorithm doesn’t guarantee fairness.

Even simpler is using two separate service instances, each with their own queue and quota. Sometimes, two is better than one.

Don’t outsource control

The irony of the blow-up with the bank’s integrator (they were not being friendly) was that in the cloud, creating another service instance costs you essentially nothing, as the cloud provider manages all the multi-tenancy for you. It’s a classic case of [designing to past constraints that no longer apply](#).

Second, it shows the dangers of excessive outsourcing. The system integrator was well aware of the problem, but preferred to pass the blame to the cloud provider, as opposed to fixing the problem. You could call it "saving face" or "being lazy", or some unfortunate variant of Conway's Law: splitting one problem into multiple parties creates new problems.

Hence my advice to large organizations:

Don't outsource thinking (or architecture decisions)

Learn from the real world

To add insult to injury, the Asian bank could have just stepped around the corner and investigated this fine piece of isolated infrastructure, aka multi-single-tenancy:



You can almost hear the OPT messages rush through the top pipe while marketing message come in loud, popping bursts. Once again, real architects learn from the real world.

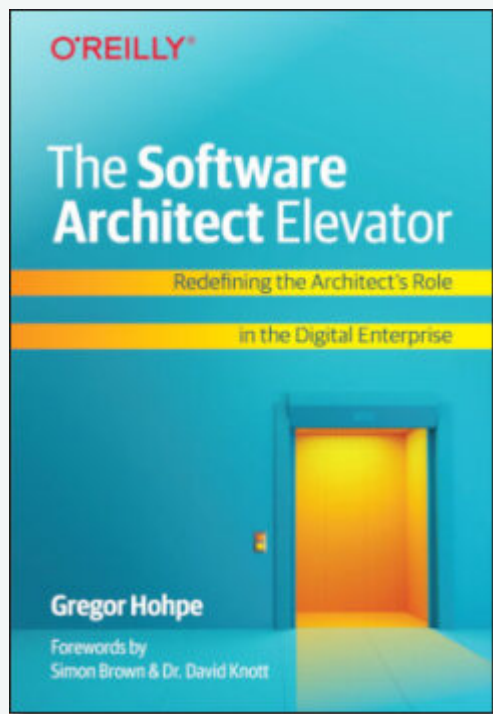
SHARE ON

 Twitter

 Facebook

 LinkedIn

Grow Your Impact as an Architect



The Software Architect Elevator helps architects and IT professionals to take their role to the next level. By sharing the real-life journey of a chief architect, it shows how to influence organizations at the intersection of business and technology. Buy it on [Amazon US](#), [Amazon UK](#), [Amazon Europe](#)

YOU MAY ALSO ENJOY



The digital grass isn't greener. It isn't grass.

Posted: Feb 27, 2026



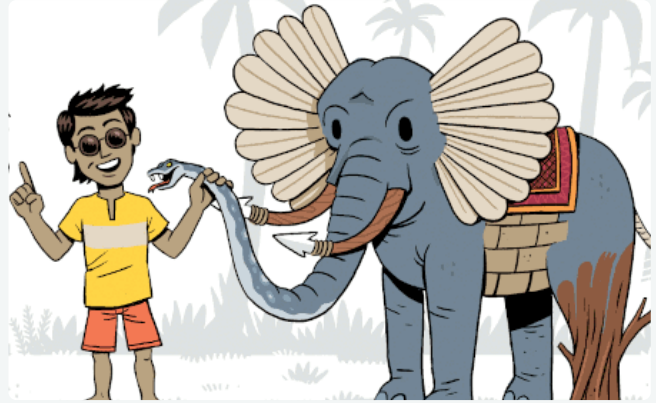
Executive Impact = Logos × Pathos

Posted: Feb 16, 2026



Invest Your Political Capital

Posted: Feb 04, 2026



The Mighty Metaphor

Posted: Jan 27, 2026

FOLLOW: [TWITTER](#)

[FEED](#)

CONTACT: [LINKEDIN](#)

[E-MAIL](#)

© 2026 Gregor Hohpe. All opinions my own.